



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

Modelo de Classes - sprint 03

Designer de Sistemas

Equipe 03 - Jogo Desconecta

Ana Lúcia de Souza - 5084

Florestal

2025

Sumário

Back-end.....	3
Classes de Entidade (Entity):.....	3
Classes de Controle:.....	4
Classes de acesso ao Banco de Dados:.....	5
Relacionamento entre as classes.....	5
Front-end.....	5

Back-end

A figura 1 mostra o Diagrama de Classes completo para os Casos de Uso da Sprint 02, isto é, **CSU01: JogarFase**. Observe que esse diagrama utiliza a categorização BCE (Boundary, Control, Entity), a qual implica que cada objeto é especialista em realizar um dos três tipos de tarefa, a saber: comunicar-se com atores (fronteira), manter as informações (entidade) ou coordenar a realização de um caso de uso (controle).

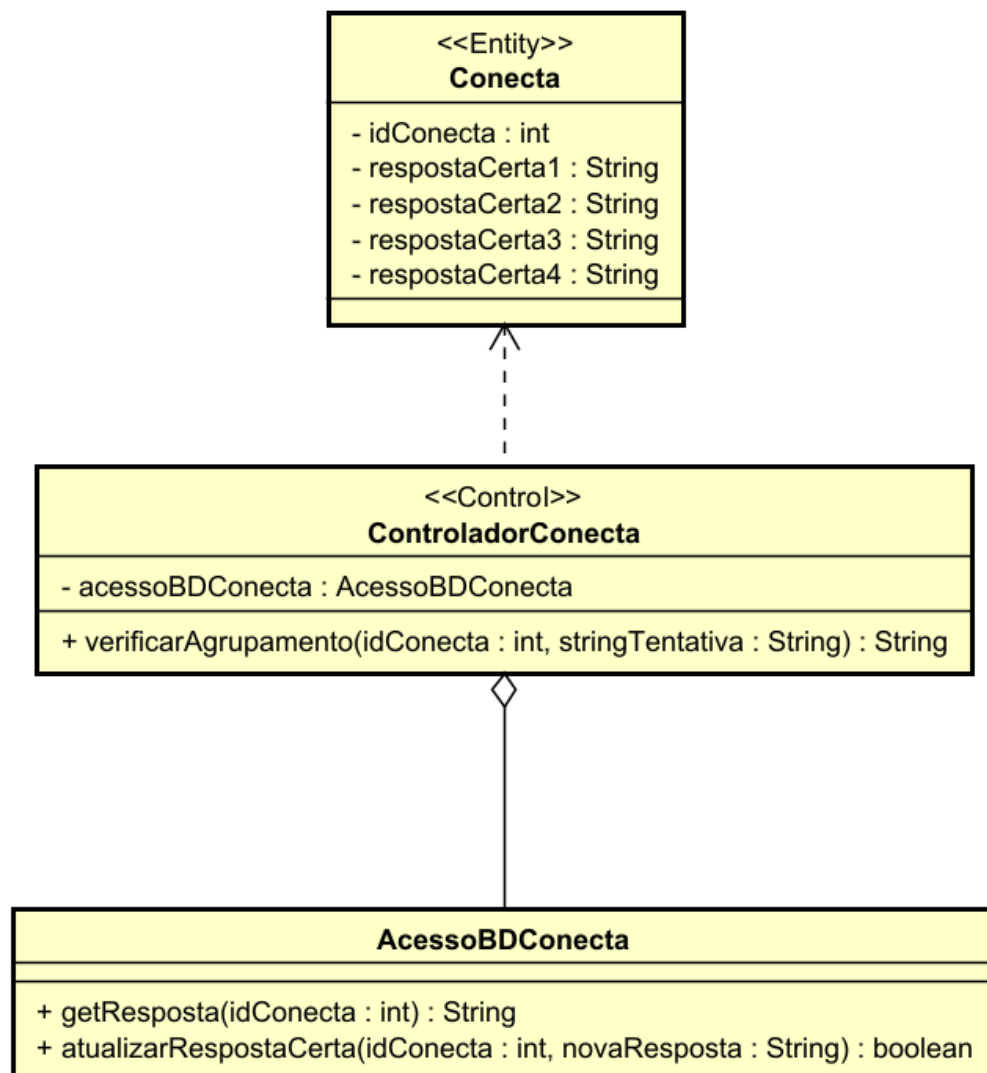


Figura 1. Diagrama de Classes para CSU06.

Classes de Entidade (Entity):

A seguir será explicada a única classe de entidade deste caso de uso:

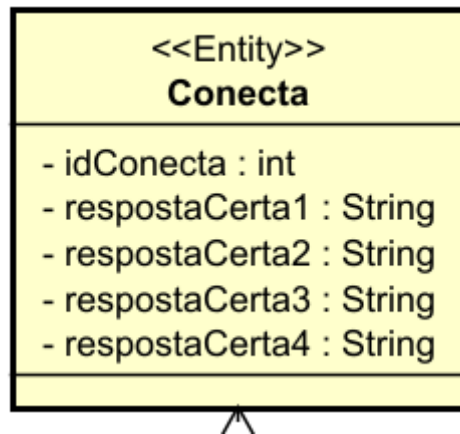


Figura 2. Classe Conecta.

- **Conecta:** Essa classe possui cinco argumentos privados como mostrado na Figura 2. O atributo “**idConecta**”, é o identificador do jogo Conecta, assim, se, por exemplo, as ilhas Dadolândia, Ciências e Matemática possuem esse tipo de desafio, então cada um deles deve ter um identificador diferente a fim de diferenciá-los. Já os atributos “**RespostaCerta**” são os responsáveis por conter as respostas certas associadas ao jogo Conecta em questão. A escolha de se utilizar quatro atributos para guardar tais strings ao invés de se utilizar uma lista ou um vetor, por exemplo, foi feita unicamente com o intuito de evitar mais uma tabela no banco de dados. Os métodos getter e setters podem ser implementados conforme a necessidade do programador.

Classes de Controle:

A classe “**ControladorConecta**”, possui o atributo privado **acessoBDConecta**, que será criado pelo construtor dessa classe. Nesta classe existe o seguinte método:

- **verificarAgrupamento(idConecta: int, stringTentativa: String) : String:** Esse método tem como objetivo verificar se uma resposta fornecida pelo aluno está correta ou não, ou seja, se a string formada pelas posições dos elementos que compõe o jogo Conecta no front-end, se constitui como uma das strings que representa a resposta certa para esse desafio específico. Para isso, esse método deve utilizar o método **getResposta()** da classe **AcessoBDConecta**. Desse modo, depois que **getResposta()** retornar as strings detentoras das respostas do desafio, o método **verificarAgrupamento()** deve comparar a string - vinda do front - com as strings presentes no banco de dados. Havendo um “casamento” entre a string formada pelo aluno e uma das strings existentes no banco, o método deve retornar uma mensagem indicando que a resposta está correta, caso contrário deve

retornar uma mensagem indicando que está errada. Observe que mesmo que este método retorne uma mensagem positiva, isso não significa que o desafio foi concluído, pois é necessário que todos os agrupamentos tenham sido feitos de forma correta. Sendo assim, também será necessário contabilizar quantas respostas certas foram obtidas a fim de descobrir se o aluno terminou com sucesso o desafio ou não, e isso deve ser feito no front-end.

Classes de acesso ao Banco de Dados:

A classe **AcessoBDConecta** será a responsável pelo acesso e manipulação do banco de dados referente à tabela Conecta. Seus métodos serão descritos a seguir:

- **getResposta(idConecta: int): String**: Esse método recebe o id do Conecta e consulta o banco de dados para obter o conjunto de respostas certas.
- **atualizarRespostaCerta(idConecta : int, novaResposta : String) : boolean**: Esse método possui apenas o objetivo de encapsular a lógica para alterar uma das respostas do desafio presentes no banco de dados, caso isso seja necessário em algum momento.

Relacionamento entre as classes

Observe que a classe **ControladorConecta** apresenta um relacionamento de agregação com **AcessoBDConecta**, sendo a primeira o objeto "todo" e a segunda o objeto "parte". A agregação representa que o controlador "tem um" acesso ao banco de dados para cumprir sua função. Em relação a **ControladorConecta**, há uma relação de dependência com a classe **Conecta**, pois o método "verificarAgrupamento()" usa o "idConecta" como parâmetro.

Front-end

Para a modelagem do front-end, foi utilizado o diagrama de máquina de estados conforme apresentado na figura 3. Nesse diagrama, **os estados representam as telas e as transições representam os botões**.

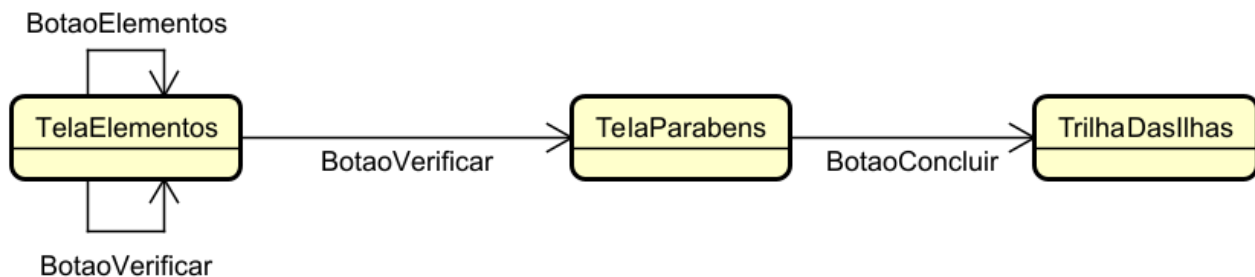


Figura 3. Modelagem front-end.

A figura 4 representa a “TelaElementos” do desafio Conecta. A ação do aluno de selecionar os elementos é feita pelo “BotaoElementos”, ou seja, cada elemento mostrado na tela da figura 4 é um botão clicável. Depois que os alunos selecionarem os elementos aos quais julgarem corretos, ele deve clicar no “BotaoVerificar” para que o sistema possa validar sua jogada ou não. Essa mesma ação será repetida até que o aluno selecione (agrupe) todos os elementos de forma correta. Assim, quando realizar o último agrupamento correto, a ação do “BotaoVerificar” o levará a “TelaParabens”, um marco para a conclusão do desafio. Assim, ao clicar em “BotaoConcluir”, o aluno é levado de volta para a trilha de fases (Ilhas).

Uma observação: no canto superior direito há um ícone que se expande para uma barra de navegação. Nela, haverá a opção de o aluno voltar para a trilha (Tela de trilhas) e ver sua pontuação. Essa parte da interface ainda está em desenvolvimento e discussão e, por este motivo, não será representada no diagrama neste momento, mas será nos próximos.

Uma outra observação importante: as ações do front-end ainda estão em processo de desenvolvimento, assim, mudanças podem ocorrer no diagrama acima. Dessa forma, o documento do protótipo, produzido pelos designers de jogo, deve ser consultado pelos desenvolvedores.



Figura 4. Prototipagem de TelaElementos.



Figura 5. Prototipagem de TelaParabens.